

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: MLA03

COURSE CODE: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO2: *Apply Dynamic Programming methods to solve reinforcement learning problems and derive optimal policies for decision-making tasks. (BL2, BL3)*

Assessment Tool Weightage: 30%

Dr G.A. SENTHIL

QUESTIONS: 10

Total Marks: 50

Annexure A

Application - Based Questions

Duration: 2Hrs

1. Design and develop a Reinforcement Learning framework a ride-sharing company uses Reinforcement Learning to match drivers with passengers efficiently. Apply RL concepts to define the state space, action space, and reward function to minimize waiting time and improve service quality. **(5 Marks)**
2. An e-learning platform uses Reinforcement Learning to recommend personalized study materials. Recall how exploration and exploitation strategies help balance familiar and new content recommendations. **(5 Marks)**
3. Design and develop a Reinforcement Learning framework a robotic vacuum cleaner uses Reinforcement Learning to clean efficiently while avoiding obstacles. Apply the concept of policy and explain how the value function improves performance over time. **(5 Marks)**
4. A cryptocurrency trading system uses Reinforcement Learning to decide buy, sell, or hold actions. Outline the challenges in designing reward functions and explain how poor reward design can affect performance. **(5 Marks)**
5. Develop a smart grid system uses Reinforcement Learning to manage electricity distribution. Create an RL model by defining state space, action space, and reward mechanism. **(5 Marks)**

6. A healthcare monitoring system uses Reinforcement Learning to adjust medication dosages. Apply RL concepts to define states, actions, and rewards, and explain how learning improves outcomes. **(5 Marks)**

7. Design a model drone delivery system uses Reinforcement Learning for navigation. Recall how exploration and exploitation strategies influence route optimization. **(5 Marks)**

8. **Design and develop** a Reinforcement Learning framework a manufacturing robot uses Reinforcement Learning to optimize assembly operations. Apply the concept of policy and explain how value functions help improve efficiency. **(5 Marks)**

9. An online advertising system uses Reinforcement Learning to maximize click-through rates. Outline reward design challenges and explain their impact on system performance. **(5 Marks)**

10. A smart irrigation system uses Reinforcement Learning to optimize water usage. Create an RL framework by defining states, actions, and reward function. **(5 Marks)**

Code of Conduct:

I M.Vineela(192472071) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

M.Vineela

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Question 1: Design and develop a Reinforcement Learning framework a ride-sharing company uses Reinforcement Learning to match drivers with passengers efficiently. Apply RL concepts to define the state space, action space, and reward function to minimize waiting time and improve service quality.

Question Meaning:

A ride-sharing app (like Uber or Ola) wants to use RL to assign the best driver to each passenger.

The question asks you to:

- Define State Space
- Define Action Space
- Define Reward Function
- Explain how RL reduces waiting time.

Explanation

- **State:** Driver location, passenger location, traffic.
- **Action:** Choose the best driver.
- **Reward:** Less waiting time.
- **Goal:** Improve customer satisfaction.

Code

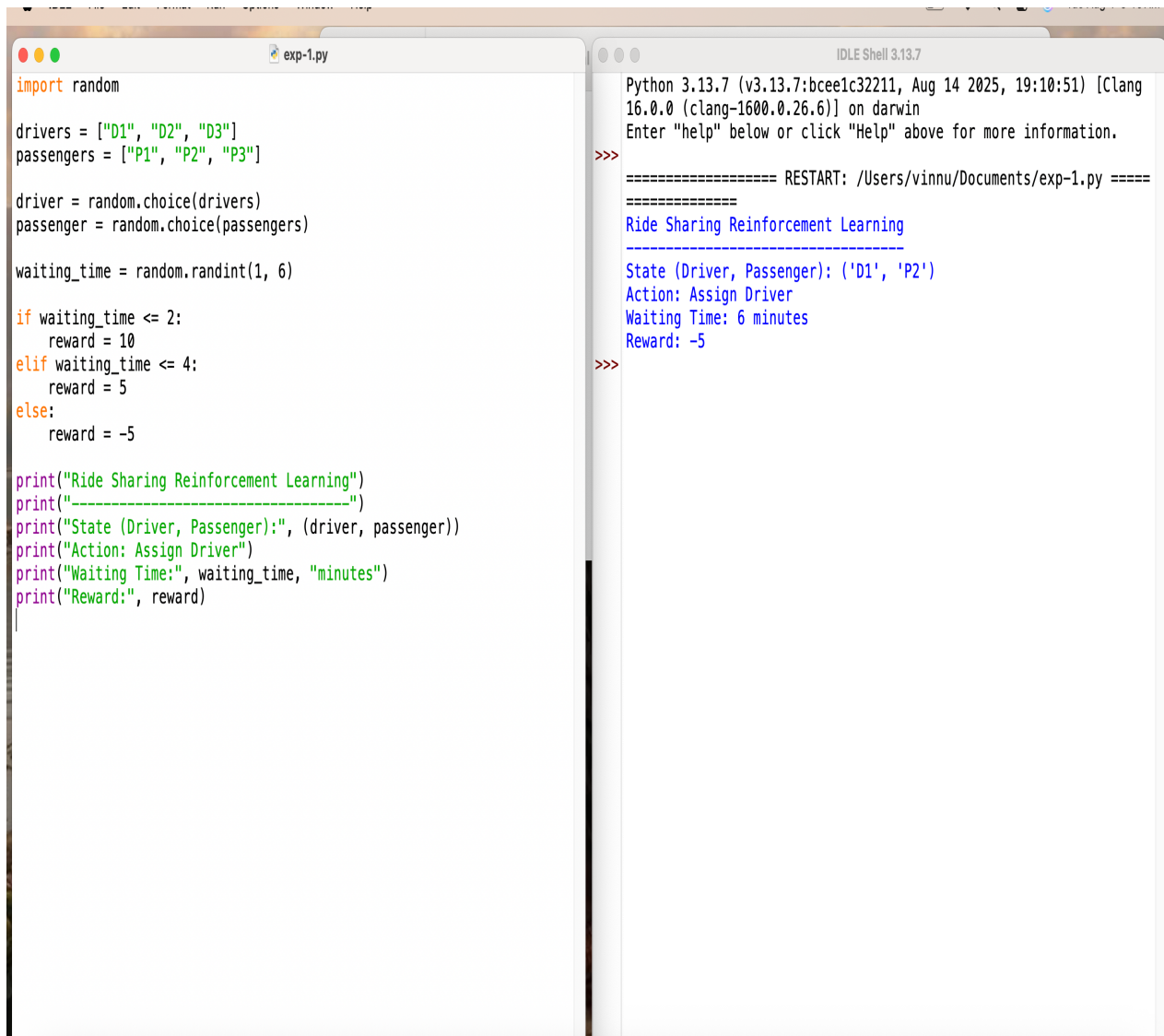
```
import random

drivers = ["D1", "D2", "D3"]
passengers = ["P1", "P2", "P3"]
driver = random.choice(drivers)
passenger = random.choice(passengers)
waiting_time = random.randint(1, 6)

if waiting_time <= 2:
    reward = 10
elif waiting_time <= 4:
    reward = 5
else:
    reward = -5

print("Ride Sharing Reinforcement Learning")
print("-----")
print("State (Driver, Passenger):", (driver, passenger))
print("Action: Assign Driver")
print("Waiting Time:", waiting_time, "minutes")
print("Reward:", reward)
```

Output:



```
exp-1.py
import random

drivers = ["D1", "D2", "D3"]
passengers = ["P1", "P2", "P3"]

driver = random.choice(drivers)
passenger = random.choice(passengers)

waiting_time = random.randint(1, 6)

if waiting_time <= 2:
    reward = 10
elif waiting_time <= 4:
    reward = 5
else:
    reward = -5

print("Ride Sharing Reinforcement Learning")
print("-----")
print("State (Driver, Passenger):", (driver, passenger))
print("Action: Assign Driver")
print("Waiting Time:", waiting_time, "minutes")
print("Reward:", reward)
```

```
IDLE Shell 3.13.7
Python 3.13.7 (v3.13.7:bcee1c32211, Aug 14 2025, 19:10:51) [Clang
16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: /Users/vinnu/Documents/exp-1.py =====
Ride Sharing Reinforcement Learning
-----
State (Driver, Passenger): ('D1', 'P2')
Action: Assign Driver
Waiting Time: 6 minutes
Reward: -5
>>>
```

Question 2: An e-learning platform uses Reinforcement Learning to recommend personalized study materials. Recall how exploration and exploitation strategies help balance familiar and new content recommendations.

Explain:

- Exploration
- Exploitation

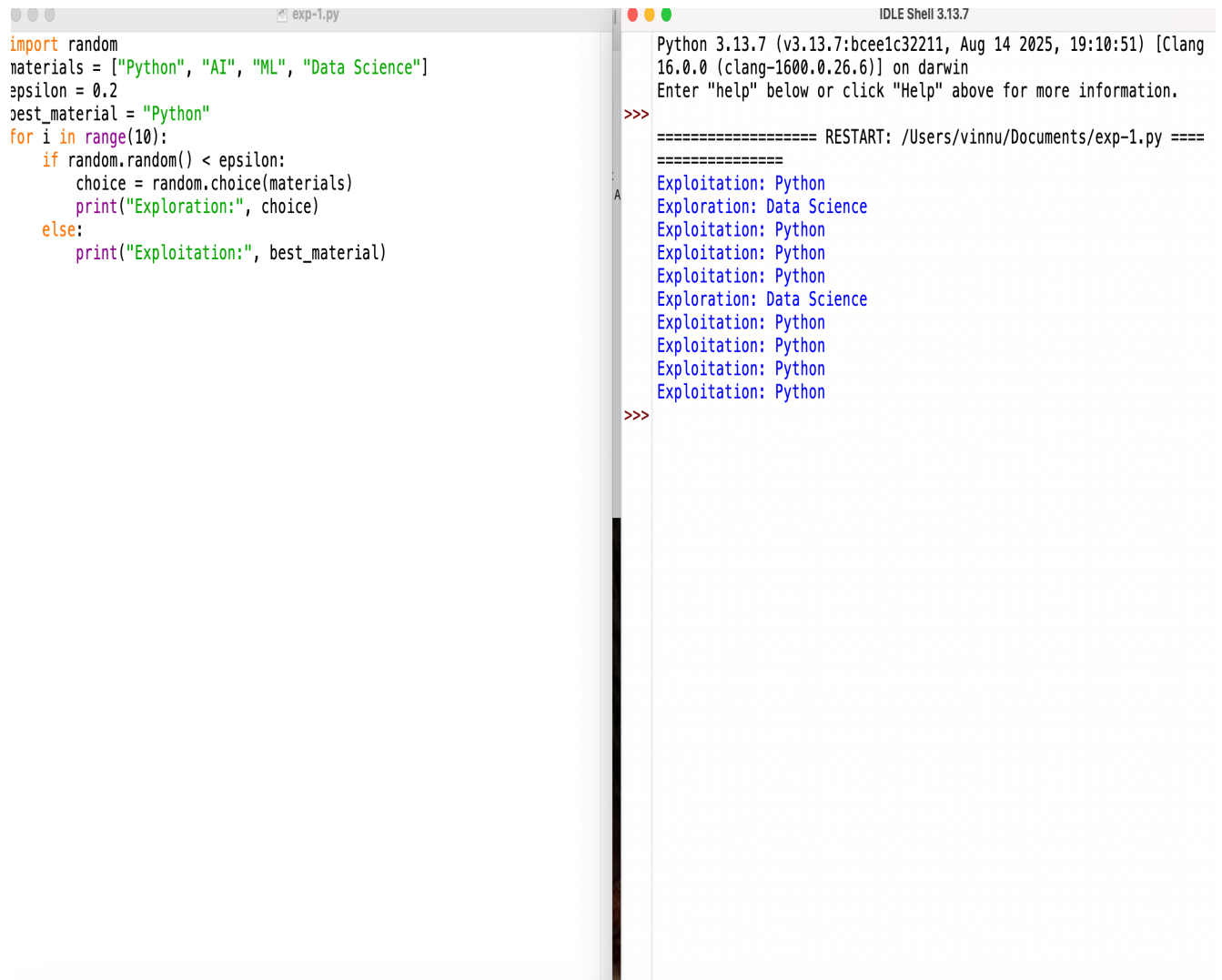
Explanation

- Exploration → Try new lessons.
- Exploitation → Recommend favorite lessons.
- RL balances both.

Code

```
import random
materials = ["Python", "AI", "ML", "Data Science"]
epsilon = 0.2
best_material = "Python"
for i in range(10):
    if random.random() < epsilon:
        choice = random.choice(materials)
        print("Exploration:", choice)
    else:
        print("Exploitation:", best_material)
```

Output:



```
exp-1.py
import random
materials = ["Python", "AI", "ML", "Data Science"]
epsilon = 0.2
best_material = "Python"
for i in range(10):
    if random.random() < epsilon:
        choice = random.choice(materials)
        print("Exploration:", choice)
    else:
        print("Exploitation:", best_material)
```

```
IDLE Shell 3.13.7
Python 3.13.7 (v3.13.7:bcee1c32211, Aug 14 2025, 19:10:51) [Clang
16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: /Users/vinnu/Documents/exp-1.py =====
>>>
Exploitation: Python
Exploration: Data Science
Exploitation: Python
Exploitation: Python
Exploitation: Python
Exploration: Data Science
Exploitation: Python
Exploitation: Python
Exploitation: Python
Exploitation: Python
>>>
```

Question 3: Design and develop a Reinforcement Learning framework a robotic vacuum cleaner uses Reinforcement Learning to clean efficiently while avoiding obstacles. Apply the concept of policy and explain how the value function improves performance over time.

Question Meaning: A robot vacuum cleans rooms while avoiding obstacles.

Explain:

- Policy
- Value Function

Explanation

- State → Robot position.
- Action → Move.
- Policy → Best movement.
- Value Function → Learns better cleaning paths.

Code

```
import random

actions = ["Up", "Down", "Left", "Right"]

state = "Dirty Floor"

policy = random.choice(actions)

reward = random.choice([10, 5, -5])

value = 0

value = value + reward

print("Robotic Vacuum Cleaner using RL")

print("-----")
```

```
print("State:", state)

print("Policy (Action):", policy)

print("Reward:", reward)

print("Value Function:", value)

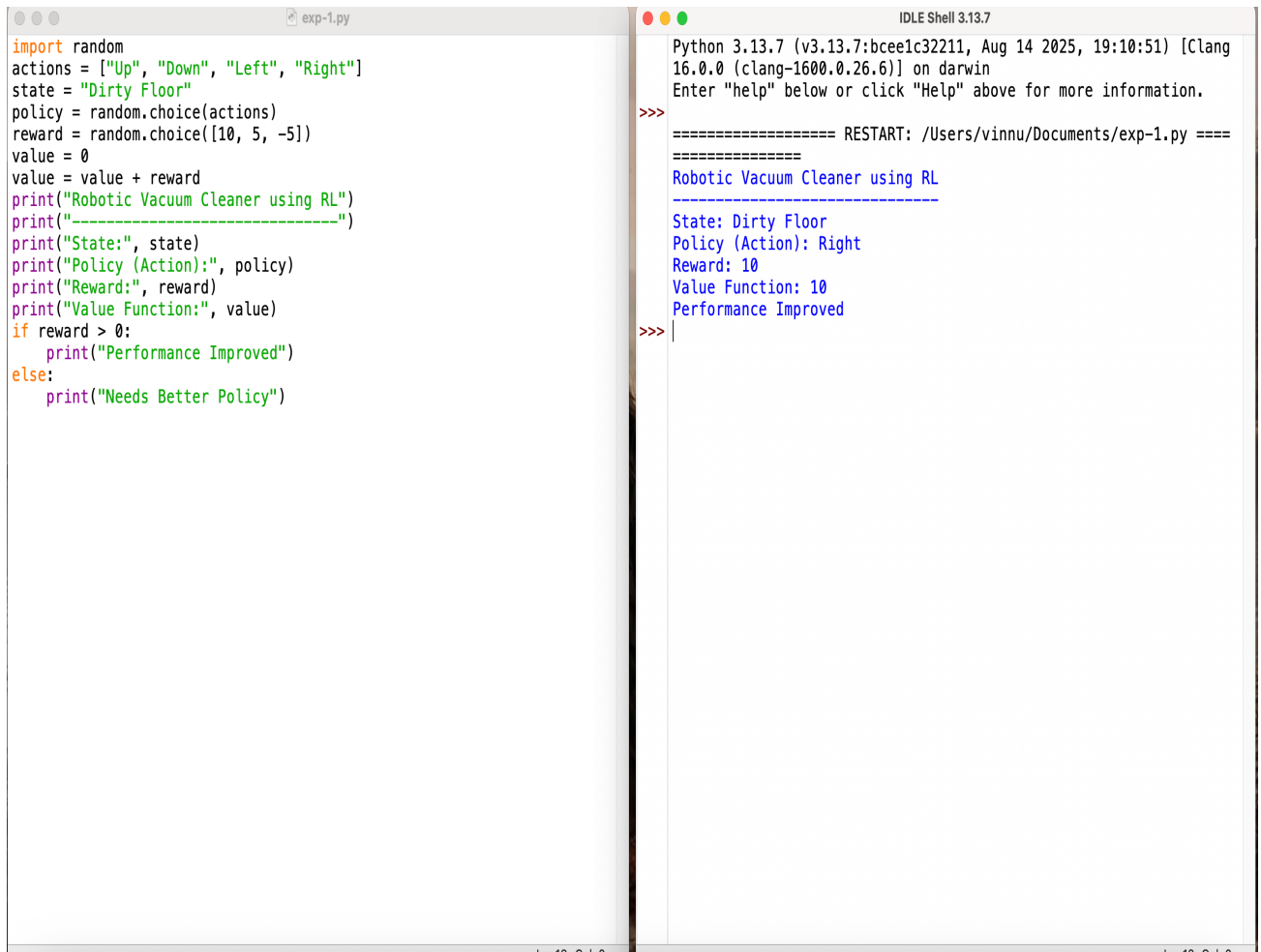
if reward > 0:

    print("Performance Improved")

else:

    print("Needs Better Policy")
```

Output:



```
exp-1.py
import random
actions = ["Up", "Down", "Left", "Right"]
state = "Dirty Floor"
policy = random.choice(actions)
reward = random.choice([10, 5, -5])
value = 0
value = value + reward
print("Robotic Vacuum Cleaner using RL")
print("-----")
print("State:", state)
print("Policy (Action):", policy)
print("Reward:", reward)
print("Value Function:", value)
if reward > 0:
    print("Performance Improved")
else:
    print("Needs Better Policy")

IDLE Shell 3.13.7
Python 3.13.7 (v3.13.7:bcee1c32211, Aug 14 2025, 19:10:51) [Clang
16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: /Users/vinnu/Documents/exp-1.py =====
Robotic Vacuum Cleaner using RL
-----
State: Dirty Floor
Policy (Action): Right
Reward: 10
Value Function: 10
Performance Improved
>>>
```

Question 4: A cryptocurrency trading system uses Reinforcement Learning to decide buy, sell, or hold actions. Outline the challenges in designing reward functions and explain how poor reward design can affect performance.

Question Meaning:

AI chooses Buy, Sell, or Hold.

Explain reward challenges.

Explanation

- Buy
- Sell
- Hold
- Reward = Profit

Code

```
import random
actions = ["Buy", "Sell", "Hold"]
state = "Market Price"
action = random.choice(actions)
reward = random.choice([10, 5, -5])
print("Cryptocurrency Trading using RL")
print("-----")
print("State:", state)
print("Action:", action)
print("Reward:", reward)
if reward > 0:
    print("Good Trading Decision")
else:
    print("Poor Trading Decision")
```

Output

```
import random

actions = ["Buy", "Sell", "Hold"]

state = "Market Price"

action = random.choice(actions)

reward = random.choice([10, 5, -5])

print("Cryptocurrency Trading using RL")
print("-----")
print("State:", state)
print("Action:", action)
print("Reward:", reward)

if reward > 0:
    print("Good Trading Decision")
else:
    print("Poor Trading Decision")
```

```
IDLE Shell 3.13.7

Python 3.13.7 (v3.13.7:bcee1c32211, Aug 14 2025, 19:10:51) [Clang
16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: /Users/vinnu/Documents/exp-1.py =====
Cryptocurrency Trading using RL
-----
State: Market Price
Action: Sell
Reward: 5
Good Trading Decision
>>> |
```


Question 5: Develop a smart grid system uses Reinforcement Learning to manage electricity distribution. Create an RL model by defining state space, action space, and reward mechanism.

Question Meaning:

Manage electricity efficiently.

Explanation

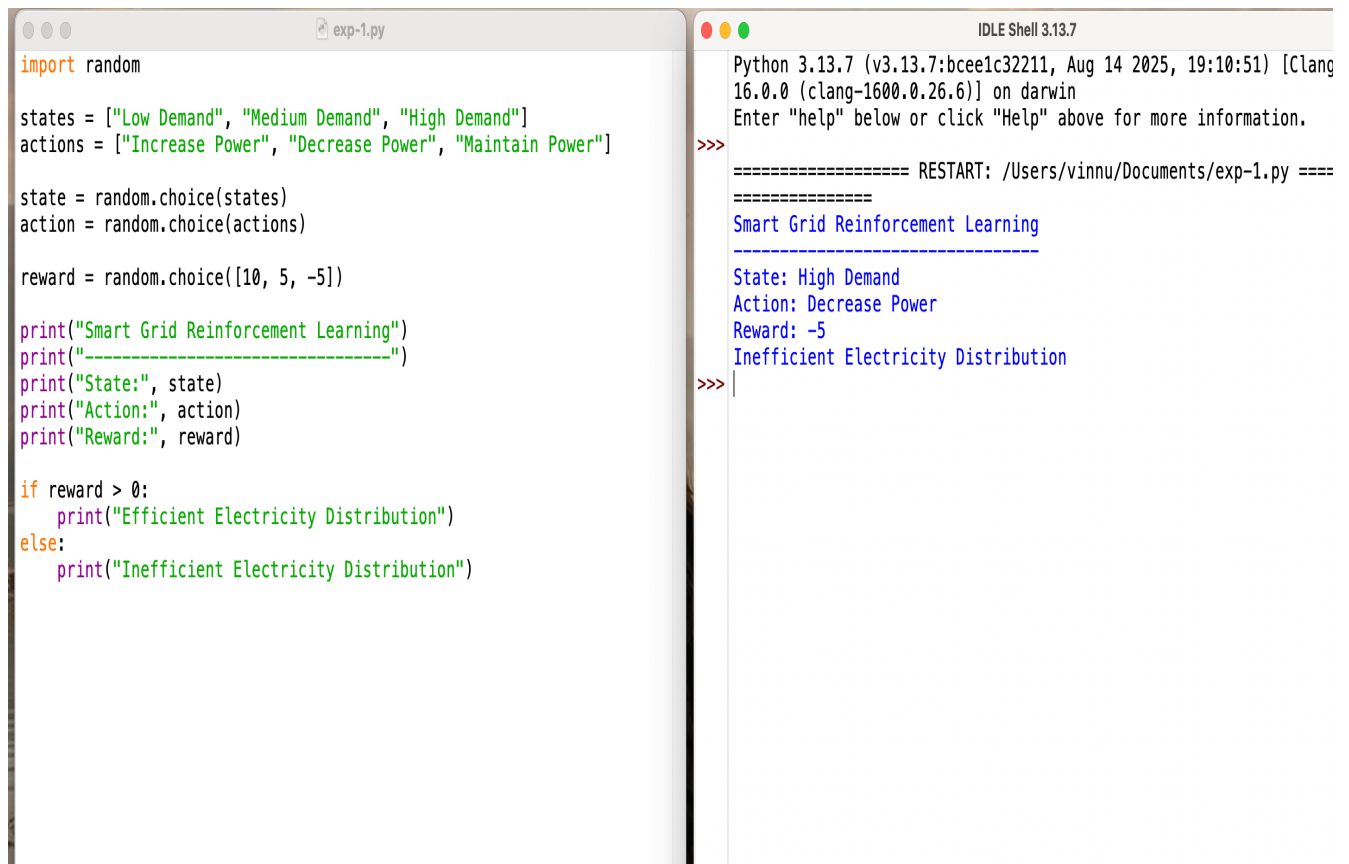
- State → Electricity demand.
- Action → Increase/Decrease supply.
- Reward → Less wastage.

Code

```
import random

states = ["Low Demand", "Medium Demand", "High Demand"]
actions = ["Increase Power", "Decrease Power", "Maintain Power"]
state = random.choice(states)
action = random.choice(actions)
reward = random.choice([10, 5, -5])
print("Smart Grid Reinforcement Learning")
print("-----")
print("State:", state)
print("Action:", action)
print("Reward:", reward)
if reward > 0:
    print("Efficient Electricity Distribution")
else:
    print("Inefficient Electricity Distribution")
```

Output:



```
import random

states = ["Low Demand", "Medium Demand", "High Demand"]
actions = ["Increase Power", "Decrease Power", "Maintain Power"]

state = random.choice(states)
action = random.choice(actions)

reward = random.choice([10, 5, -5])

print("Smart Grid Reinforcement Learning")
print("-----")
print("State:", state)
print("Action:", action)
print("Reward:", reward)

if reward > 0:
    print("Efficient Electricity Distribution")
else:
    print("Inefficient Electricity Distribution")
```

Python 3.13.7 (v3.13.7:bcee1c32211, Aug 14 2025, 19:10:51) [Clang 16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.

>>> ===== RESTART: /Users/vinnu/Documents/exp-1.py =====
Smart Grid Reinforcement Learning

State: High Demand
Action: Decrease Power
Reward: -5
Inefficient Electricity Distribution
>>> |

Question 6: A healthcare monitoring system uses Reinforcement Learning to adjust medication dosages. Apply RL concepts to define states, actions, and rewards, and explain how learning improves outcomes.

Question Meaning

Adjust medicine dosage automatically.

Explanation

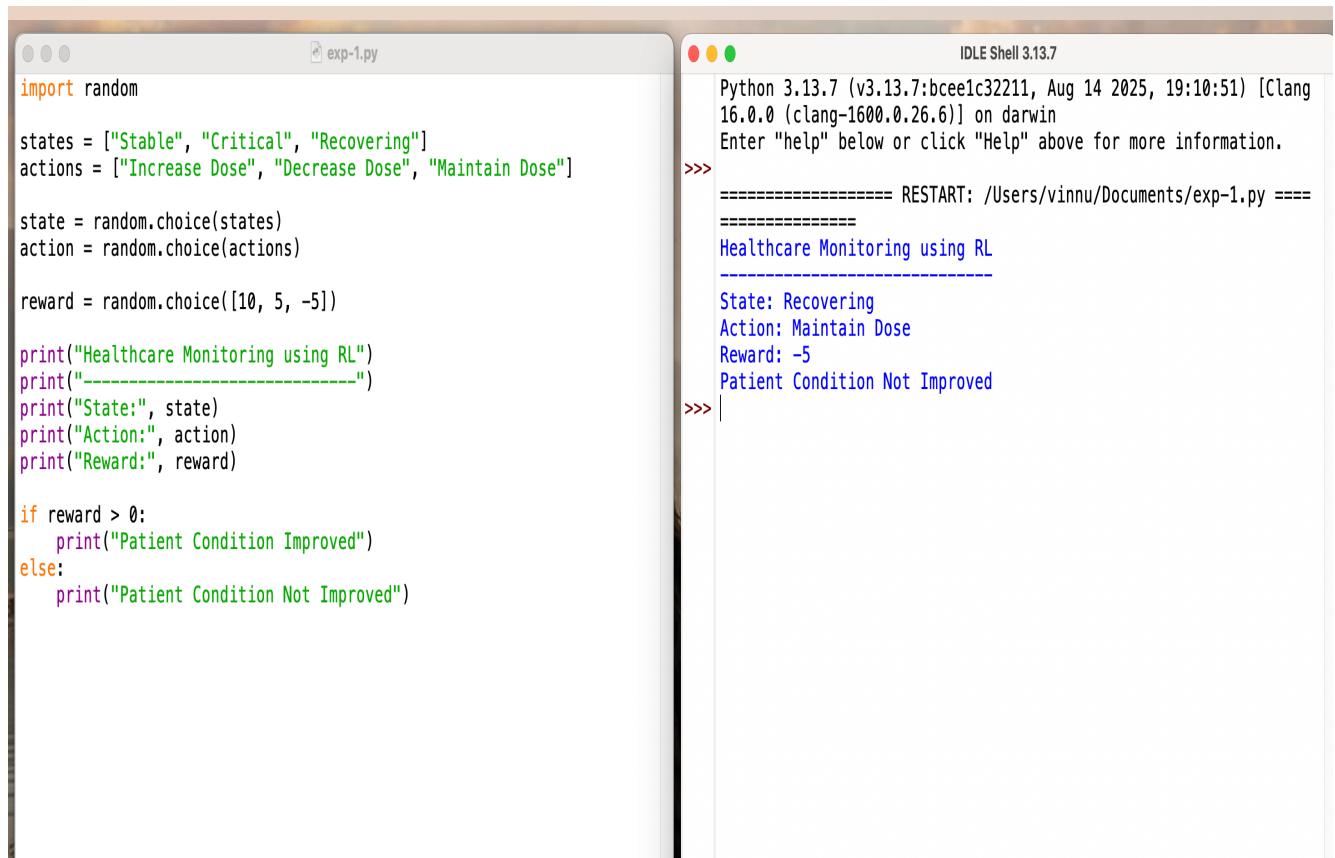
- State → Patient health.
- Action → Increase/Decrease dosage.
- Reward → Patient recovers.

Code

```
import random

states = ["Stable", "Critical", "Recovering"]
actions = ["Increase Dose", "Decrease Dose", "Maintain Dose"]
state = random.choice(states)
action = random.choice(actions)
reward = random.choice([10, 5, -5])
print("Healthcare Monitoring using RL")
print("-----")
print("State:", state)
print("Action:", action)
print("Reward:", reward)
if reward > 0:
    print("Patient Condition Improved")
else:
    print("Patient Condition Not Improved")
```

Output



The image shows a screenshot of a Python script and its output in an IDE. The script is named 'exp-1.py' and is located in the file explorer on the left. The script defines a list of states ('Stable', 'Critical', 'Recovering') and actions ('Increase Dose', 'Decrease Dose', 'Maintain Dose'). It uses the 'random' module to choose a state and an action. The script prints the state, action, and reward, and then checks if the reward is greater than 0 to print 'Patient Condition Improved' or 'Patient Condition Not Improved'.

```
import random

states = ["Stable", "Critical", "Recovering"]
actions = ["Increase Dose", "Decrease Dose", "Maintain Dose"]

state = random.choice(states)
action = random.choice(actions)

reward = random.choice([10, 5, -5])

print("Healthcare Monitoring using RL")
print("-----")
print("State:", state)
print("Action:", action)
print("Reward:", reward)

if reward > 0:
    print("Patient Condition Improved")
else:
    print("Patient Condition Not Improved")
```

The output on the right shows the execution of the script. It starts with the Python version and system information. The output is as follows:

```
>>>
===== RESTART: /Users/vinnu/Documents/exp-1.py =====
Healthcare Monitoring using RL
-----
State: Recovering
Action: Maintain Dose
Reward: -5
Patient Condition Not Improved
>>> |
```

Question 7: Design a model drone delivery system uses Reinforcement Learning for navigation. Recall how exploration and exploitation strategies influence route optimization.

Question Meaning

A delivery drone uses Reinforcement Learning to find the best route for delivering packages. The drone learns from its previous flights and improves its navigation over time.

The question asks you to explain:

- Exploration
- Exploitation
- How they help in finding the best route.

Code

```
import random

states = ["Warehouse", "In Transit", "Destination"]
actions = ["Move Left", "Move Right", "Move Forward"]

state = random.choice(states)
action = random.choice(actions)
reward = random.choice([10, 5, -5])

print("Drone Delivery using RL")
print("-----")
print("State:", state)
print("Action:", action)
print("Reward:", reward)
```

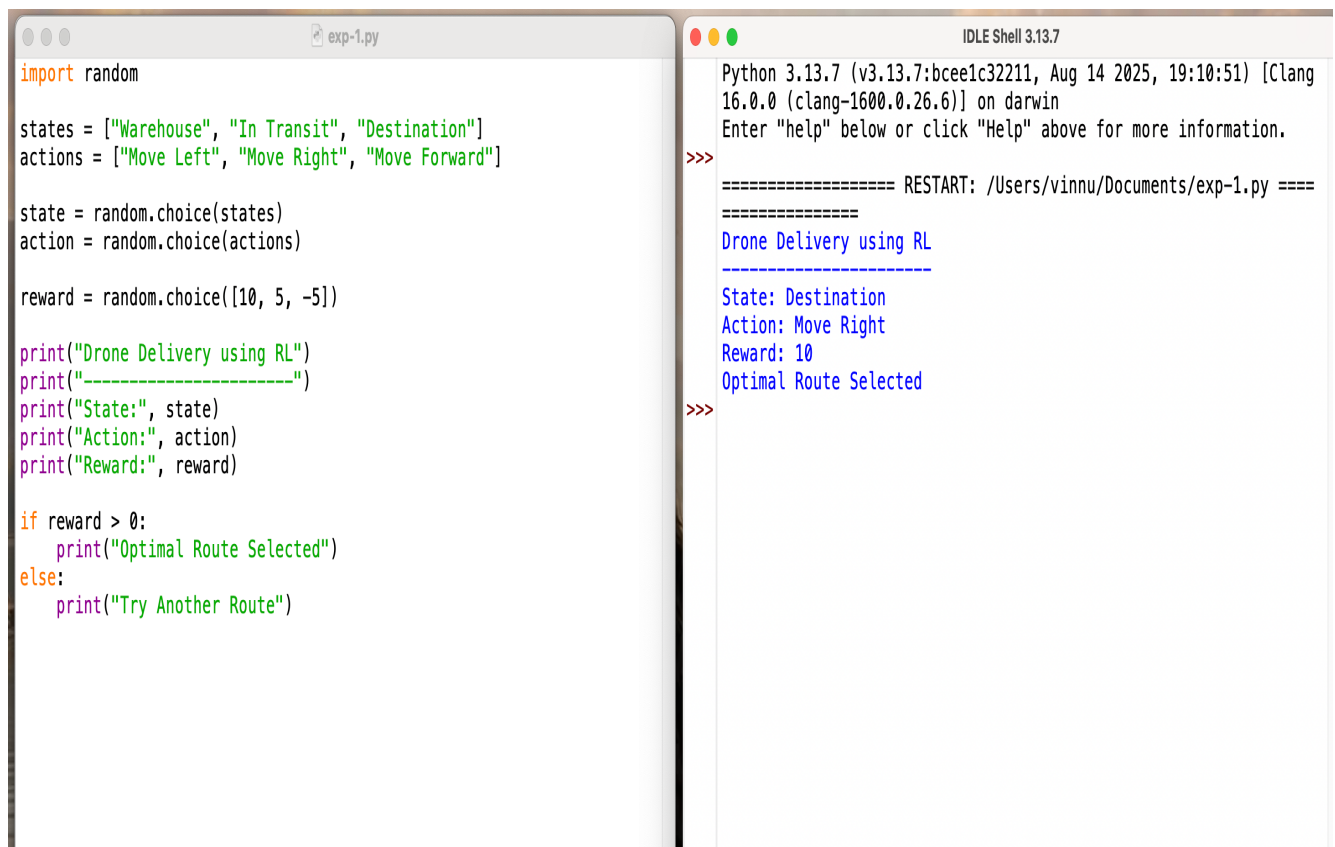
```
if reward > 0:

    print("Optimal Route Selected")

else:

    print("Try Another Route")
```

Output:



The image shows a screenshot of a Python script and its output in an IDE. The script is named 'exp-1.py' and is located in the file explorer on the left. The script content is as follows:

```
import random

states = ["Warehouse", "In Transit", "Destination"]
actions = ["Move Left", "Move Right", "Move Forward"]

state = random.choice(states)
action = random.choice(actions)

reward = random.choice([10, 5, -5])

print("Drone Delivery using RL")
print("-----")
print("State:", state)
print("Action:", action)
print("Reward:", reward)

if reward > 0:
    print("Optimal Route Selected")
else:
    print("Try Another Route")
```

The output is displayed in the IDLE Shell 3.13.7 on the right. It shows the execution of the script, including the state, action, and reward, and the final output message.

```
Python 3.13.7 (v3.13.7:bcee1c32211, Aug 14 2025, 19:10:51) [Clang
16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: /Users/vinnu/Documents/exp-1.py =====
=====
Drone Delivery using RL
-----
State: Destination
Action: Move Right
Reward: 10
Optimal Route Selected
>>>
```

Question 8: Design and develop a Reinforcement Learning framework a manufacturing robot uses Reinforcement Learning to optimize assembly operations. Apply the concept of policy and explain how value functions help improve efficiency.

The question asks you to explain:

- Policy
- Value Function
- How they improve efficiency.

Explanation

- Policy: A rule that tells the robot which action to perform next.
- Value Function: Estimates how good each action or state is based on future rewards.
- The robot learns from previous assembly tasks and selects the best actions.
- Over time, it reduces errors, increases production speed, and improves product quality.

Code

```
import random

states = ["Part Ready", "Assembly", "Completed"]
actions = ["Pick Part", "Assemble Part", "Finish"]
state = random.choice(states)
action = random.choice(actions)
reward = random.choice([10, 5, -5])
value = reward

print("Manufacturing Robot using RL")
print("-----")
print("State:", state)
```

```
print("Action:", action)

print("Reward:", reward)

print("Value Function:", value)

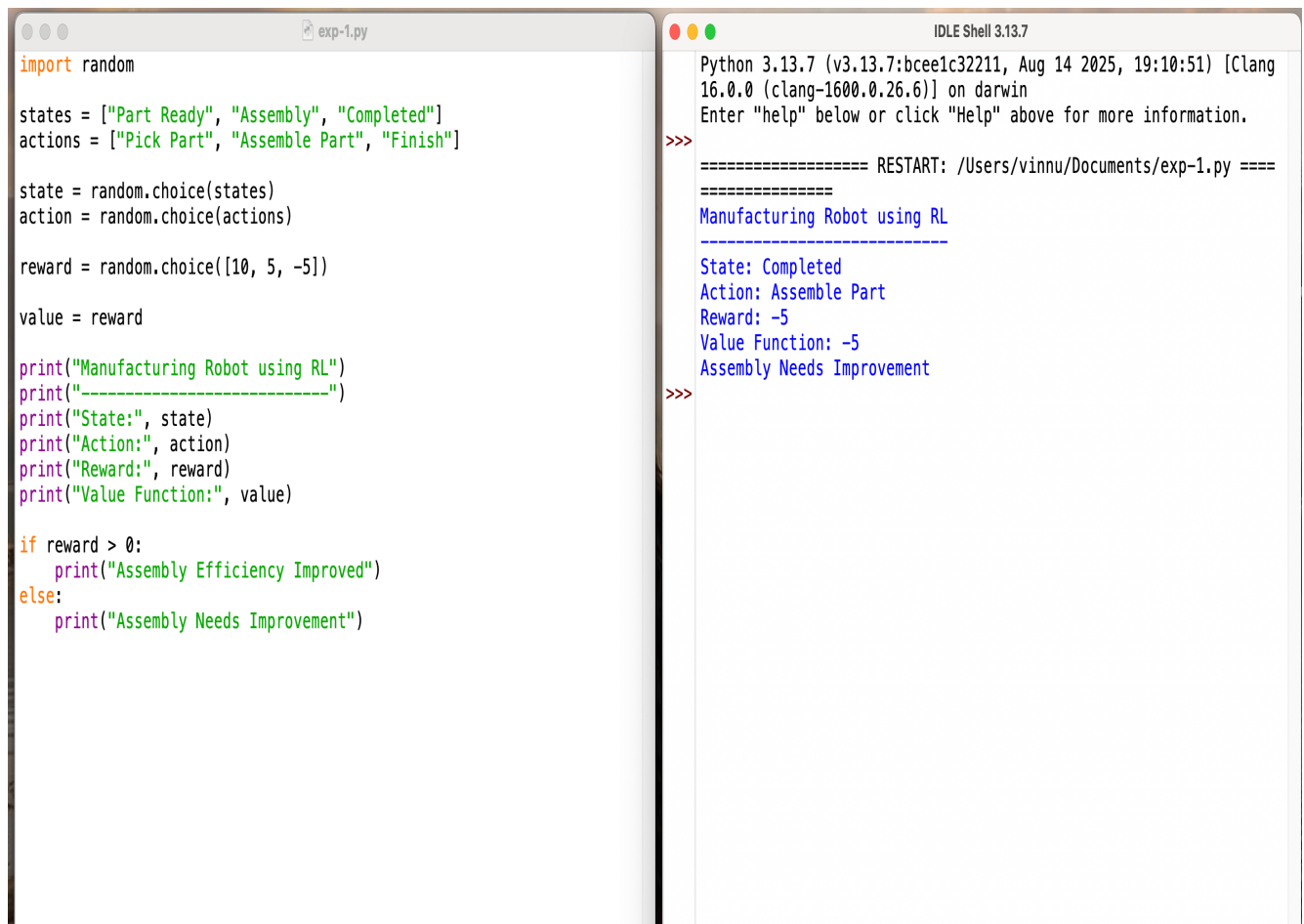
if reward > 0:

    print("Assembly Efficiency Improved")

else:

    print("Assembly Needs Improvement")
```

Output

The image shows a side-by-side comparison of a Python script and its execution output. On the left, a code editor window titled 'exp-1.py' contains a script that imports the 'random' module, defines a list of states ('Part Ready', 'Assembly', 'Completed') and actions ('Pick Part', 'Assemble Part', 'Finish'), and then randomly selects a state, action, and reward. It prints out the state, action, reward, and value function. An if-statement checks if the reward is greater than 0, printing 'Assembly Efficiency Improved' if true, and 'Assembly Needs Improvement' if false. On the right, an 'IDLE Shell 3.13.7' window shows the output of the script. It starts with the Python version and platform information, followed by a prompt to enter 'help'. The script's output is displayed, showing the state as 'Completed', action as 'Assemble Part', reward as -5, and value function as -5. Since the reward is not greater than 0, the output is 'Assembly Needs Improvement'.

```
import random

states = ["Part Ready", "Assembly", "Completed"]
actions = ["Pick Part", "Assemble Part", "Finish"]

state = random.choice(states)
action = random.choice(actions)

reward = random.choice([10, 5, -5])

value = reward

print("Manufacturing Robot using RL")
print("-----")
print("State:", state)
print("Action:", action)
print("Reward:", reward)
print("Value Function:", value)

if reward > 0:
    print("Assembly Efficiency Improved")
else:
    print("Assembly Needs Improvement")
```

```
Python 3.13.7 (v3.13.7:bcee1c32211, Aug 14 2025, 19:10:51) [Clang
16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: /Users/vinnu/Documents/exp-1.py =====
Manufacturing Robot using RL
-----
State: Completed
Action: Assemble Part
Reward: -5
Value Function: -5
Assembly Needs Improvement
>>>
```


Question 9: An online advertising system uses Reinforcement Learning to maximize click-through rates. Outline reward design challenges and explain their impact on system performance.

Question Meaning

An online advertising system uses Reinforcement Learning to decide which advertisement should be shown to users. The goal is to increase the number of user clicks.

Explanation

- **Reward:** A positive reward is given when a user clicks an advertisement.
- **Challenge:** Not every click means the advertisement was useful or resulted in a purchase.
- If rewards are designed poorly, the system may repeatedly show irrelevant or misleading ads.
- improve user experience, and increase click-through rates over time.

Code

```
import random

states = ["User Online", "Ad Displayed", "Ad Clicked"]
actions = ["Show Ad A", "Show Ad B", "Show Ad C"]
state = random.choice(states)
action = random.choice(actions)
reward = random.choice([10, 5, -5])
print("Online Advertising using RL")
print("-----")
print("State:", state)
print("Action:", action)
print("Reward:", reward)
```

```
if reward > 0:
    print("Higher Click-Through Rate")
else:
    print("Lower Click-Through Rate")
```

Output

```
import random

states = ["User Online", "Ad Displayed", "Ad Clicked"]
actions = ["Show Ad A", "Show Ad B", "Show Ad C"]

state = random.choice(states)
action = random.choice(actions)

reward = random.choice([10, 5, -5])

print("Online Advertising using RL")
print("-----")
print("State:", state)
print("Action:", action)
print("Reward:", reward)

if reward > 0:
    print("Higher Click-Through Rate")
else:
    print("Lower Click-Through Rate")
```

```
Python 3.13.7 (v3.13.7:bcee1c32211, Aug 14 2025, 19:10:51) [Clang
16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: /Users/vinnu/Documents/exp-1.py =====
Online Advertising using RL
-----
State: User Online
Action: Show Ad C
Reward: 5
Higher Click-Through Rate
>>>
```

Question 10: A smart irrigation system uses Reinforcement Learning to optimize water usage. Create an RL framework by defining states, actions, and reward function.

Question Meaning

Water crops using minimum water.

Explanation

- **State:** Soil moisture level, weather condition, and temperature.
- **Action:** Increase watering, decrease watering, or stop watering.
- **Reward:** Positive reward when crops receive enough water with minimum water wastage.
- Over time, the RL system learns the best watering schedule, improves crop growth, and conserves water.

Code

```
import random

states = ["Dry Soil", "Normal Soil", "Wet Soil"]

actions = ["Increase Water", "Decrease Water", "Maintain Water"]

state = random.choice(states)

action = random.choice(actions)

reward = random.choice([10, 5, -5])

print("Smart Irrigation using RL")

print("-----")

print("State:", state)

print("Action:", action)
```

```
print("Reward:", reward)
```

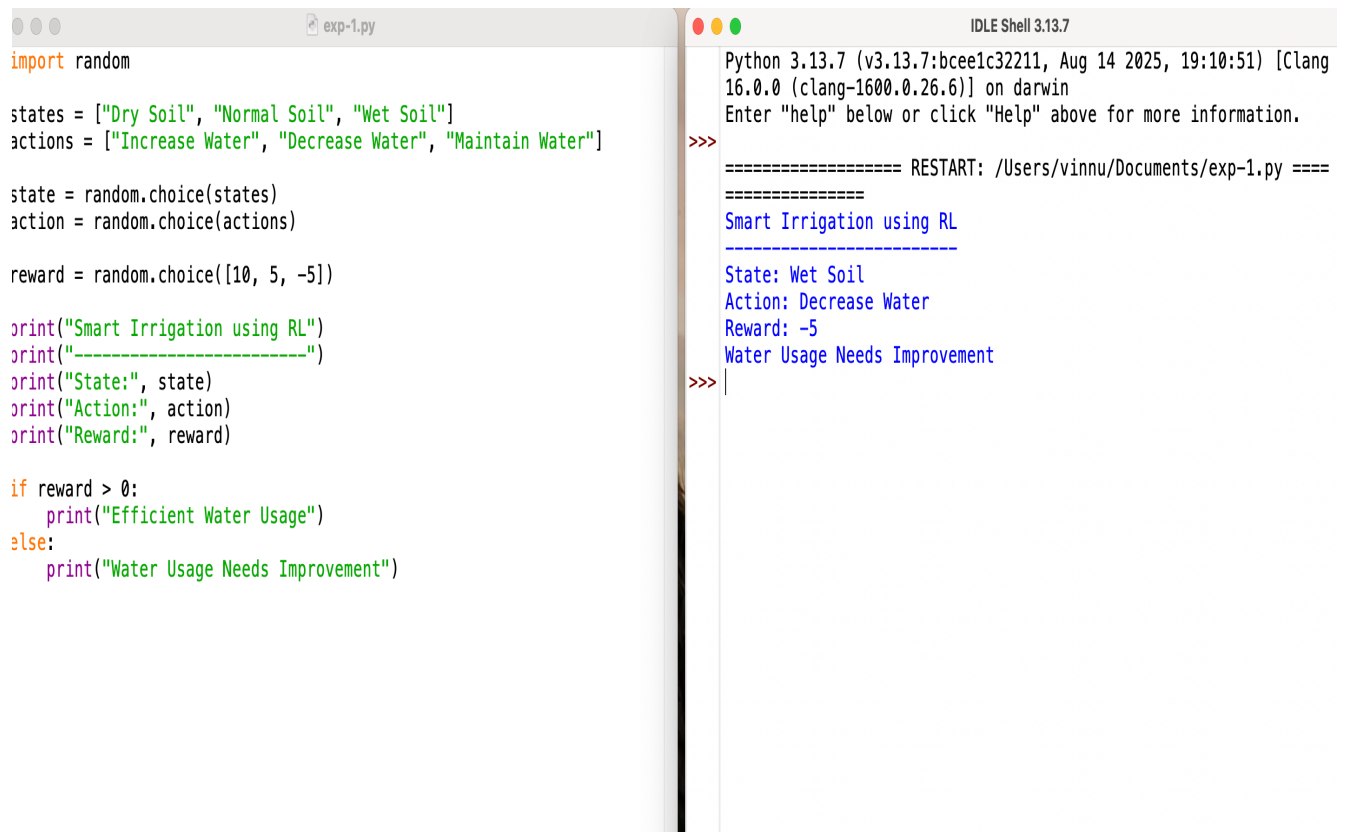
```
if reward > 0:
```

```
    print("Efficient Water Usage")
```

```
else:
```

```
    print("Water Usage Needs Improvement")
```

Output:



```
import random

states = ["Dry Soil", "Normal Soil", "Wet Soil"]
actions = ["Increase Water", "Decrease Water", "Maintain Water"]

state = random.choice(states)
action = random.choice(actions)

reward = random.choice([10, 5, -5])

print("Smart Irrigation using RL")
print("-----")
print("State:", state)
print("Action:", action)
print("Reward:", reward)

if reward > 0:
    print("Efficient Water Usage")
else:
    print("Water Usage Needs Improvement")
```

```
Python 3.13.7 (v3.13.7:bcee1c32211, Aug 14 2025, 19:10:51) [Clang
16.0.0 (clang-1600.0.26.6)] on darwin
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: /Users/vinnu/Documents/exp-1.py =====
Smart Irrigation using RL
-----
State: Wet Soil
Action: Decrease Water
Reward: -5
Water Usage Needs Improvement
>>>
```

